

Assignment: Problem Statement · Lab 2

1 Scenario

You are building a small internal tool for a software product team. Every day the team receives dozens of messages in their issue inbox, bug reports, feature requests, unclear documentation complaints, and general questions, all mixed together in one queue. Right now, a human has to read every message and manually decide which category it belongs to and how to respond, which is slow and inconsistent.

Your job is to build an **AI-powered issue triage assistant** using LangChain that automatically reads an incoming message, works out what kind of issue it is, and drafts an appropriate first response, the same overall pattern you practiced with the reference files, applied to a new problem.

2 Your Task

Build a Python application using LangChain that:

1. Classifies an incoming issue message into exactly one of four categories:
 - `bug_report` : something in the software is broken or behaving incorrectly
 - `feature_request` : a suggestion for new or improved functionality
 - `documentation_issue` : the docs/README/guide are missing, unclear, or incorrect
 - `question` : a general how-do-I / clarification question that isn't a bug or a request
2. Routes the message to a **different prompt/template per category** and generates a contextual reply using the LLM.
3. Prints the input message, the classification result, and the generated reply clearly to the terminal.

3 Category-Specific Behaviour (required)

Each category's prompt must make the model behave differently, this is the actual point of the assignment, so make each response meaningfully distinct instead of a generic "thanks for reaching out":

Category	What the reply should do
<code>bug_report</code>	Acknowledge the bug and ask for reproduction steps, environment/version details, and expected vs. actual behaviour if not already given.
<code>feature_request</code>	Thank the user, ask what problem the feature would solve or how they'd use it, and note it will be considered by the team.
<code>documentation_issue</code>	Acknowledge the documentation gap, ask which page/section was unclear, and thank them for helping improve the docs.
<code>question</code>	Answer helpfully and directly if possible, or point them to where they can find the answer.

4 Deliverables

- A single Python file implementing the full pipeline (classification + routing + reply generation).
- Four screenshots, one run per category, each showing input, classification result, and generated reply.